End-to-end claim case

From an opaque, manually coordinated claims workflow to a visible, rule-driven process supported by BPMN 2.0, Excel and SQL.

PROJECT TYPE	**Academic case study**
BUSINESS PROCESS	**Customer claims management**
FOCUS	**Process visibility, data reliability and refund automation**

Project reference: https://wero-git.github.io/porto-folio/claims-case-study.html

WE

Winston Engamba

Digitalization - Process Optimization - Management

MAY 2026

01 / EXECUTIVE OVERVIEW

Making the claims workflow visible and measurable

The project analyses a claims process from registration to resolution, formalizes its decision logic and connects process execution with structured operational data.

4 Core process stages	**3** Challenge themes	**4** Improvement steps	**3** Core tools applied

Project objective

The objective is not simply to redraw the process. It is to establish a reliable connection between activities, responsibilities, decisions and the data required to calculate and document each refund.

- Document the end-to-end flow from complaint receipt to customer notification.
- Expose bottlenecks, handovers, exceptional cases and time-dependent decisions.
- Translate refund rules into reproducible Excel formulas and SQL logic.
- Create a structured foundation for reporting, automation and continuous improvement.

Scope and evidence basis

This report consolidates the portfolio case-study page, the initial and optimized BPMN models, the process overview, the Excel prototype and the SQL result preview. The dataset is an academic test set; operational effects are therefore presented separately from projected benefits.

Report structure

02 / INITIAL SITUATION

A sequential process dependent on manual coordination

The initial workflow involved Customer Service, Technical Service, Accounting, the warehouse, the customer and, in exceptional cases, the Legal Department.

01
Registration
Customer complaint is received and recorded.

02
Assessment
Customer Service checks legitimacy and return eligibility.

03
Decision
Technical condition and issue source determine the refund.

04
Resolution
Accounting pays, product is stored and customer is notified.

Initial operating logic

- Customer Service evaluates the complaint. A legitimate request starts the return process; a malicious or slanderous case is escalated to Legal and the refund flow stops.
- For an accepted return, the customer receives a request to return the product within 30 days. If the product is not received, the claim must be aborted.
- Technical Service evaluates product condition and documents the result before warehouse storage.
- Accounting manually interprets the evaluation and calculates a full or partial refund before the final customer notification.

Decision point	Possible outcome	Operational consequence
Claim legitimacy	Legitimate / malicious	Return flow begins or the case is transferred to Legal.
Return deadline	Received / not received	Technical assessment continues or the process is aborted after 30 days.
Issue source	Customer / production / unknown	Determines responsibility and contributes to the refund rule.
Product condition	Good / damaged / very damaged	Determines the applicable refund rate together with issue source.

Key dependency

Technical assessment and accounting execution were connected by a manual spreadsheet handover. The refund could not be completed reliably until another department had read and interpreted the latest file entries.

03 / PROBLEMS OBSERVED

Four structural issues behind three challenge themes

The site groups the challenge into process visibility, information flow and processing efficiency. The detailed audit reveals four concrete structural issues.

01 / HANDOVER

Sequential data handover

Technical staff updated Excel and Accounting later consulted the file. This created waiting time, version uncertainty and reading errors.

02 / RULES

Complex refund rules

The 100%, 85% and 70% rates depended on combinations of issue source and product condition, making manual interpretation fragile.

03 / EXCEPTIONS

Invisible legal cases

Claims transferred to Legal left the standard spreadsheet flow, breaking end-to-end visibility and ownership.

04 / TIME

Manual 30-day control

No automatic signal identified overdue returns. Staff had to review rows manually to find cases that should be aborted.

Observed symptom	Root cause	Likely impact
Late refund processing	Accounting waits for and rereads technical entries	Longer lead time and customer dissatisfaction
Inconsistent reimbursement	Business rules interpreted manually	Financial error and unequal case treatment
Lost exceptional cases	Legal branch tracked outside the main dataset	Weak accountability and incomplete reporting
Expired claims remain open	No time-fencing or automated status flag	Manual workload and incorrect process status

Root-cause pattern

The process problem was not caused by a single task. It resulted from a weak connection between process logic and data logic: decisions existed in practice but were not encoded in a shared structure that all departments could use consistently.

Design requirement

The improved process had to make every normal, exceptional and time-dependent path explicit while reducing the need for manual interpretation between departments.

04 / SOLUTIONS PROPOSED

From process analysis to rule-driven execution

The redesigned solution combines formal BPMN modeling, structured data and automated decision logic rather than treating them as separate outputs.

01 / UNDERSTANDING

Process analysis

Document activities, roles, events and decision points from complaint receipt through closure.

02 / VISUALIZATION

BPMN 2.0 modeling

Represent sequence flows, gateways, legal escalation, return deadline and cross-department responsibilities.

03 / ORGANIZATION

Data structuring

Create consistent claim records containing issue source, product condition, refund rate, amount and process status.

04 / IMPROVEMENT

Automation logic

Calculate refunds and status outcomes automatically, reducing non-value interpretation and manual handovers.

Optimized claim-handling flow

01 **Validate**
Register and classify the complaint.

02 **Control time**
Track the 30-day return window and status.

03 **Assess once**
Record condition and issue source in structured fields.

04 **Execute rule**
Apply 100%, 85%, 70% or pending outcome.

Before	After
Manual transfer from technical staff to Accounting	Shared structured record usable by Excel and SQL
Refund percentage interpreted case by case	Rule-based percentage derived from defined fields

Before	After
Legal and overdue cases outside standard follow-up	Explicit statuses and branches retained in the model
Process known mainly through individual practice	BPMN model provides common end-to-end visibility

Parallelization principle

After the technical evaluation is recorded, warehouse storage and refund execution can proceed from the same validated information. This removes the need to recalculate or reinterpret the case in a separate manual step.

04.1 / RULES AND DATA

A transparent financial decision engine

The prototype translates the main business rules into Excel formulas and a relational SQL structure so that identical inputs produce identical outputs.

Issue source	Product condition	Prototype rate	Resulting status
Customer	Good	100%	Received
Production	Damaged	100%	Received
Customer	Damaged	85%	Received
Customer	Very damaged	70%	Received
Unknown	Not received	0%	Pending return

Refund amount = unit price x refund rate. The rule set above reflects the implemented prototype visible in the Excel dataset; it must be aligned with the final approved company policy before production deployment.

Structured claim record

Field	Purpose	Example
claim_id	Unique case identifier	1
customer_name	Customer reference	Alice Johnson
product_name / unit_price	Product and financial base	HP PC / EUR 1,200

Field	Purpose	Example
issue_source	Responsibility classification	Customer
product_condition	Technical evaluation result	Very_Damaged
rate / total_refunded	Calculated reimbursement	70% / EUR 840
process_status	Workflow control	Received / Pending_Return

Prototype SQL output

claim_id	customer_name	product_name	unit_price	issue_source	product_condition	rate	total_refunded	process_status
1	Alice Johnson	HP PC AMD RYZEN 2025	1200.00 €	Customer	Very_Damaged	70%	840.00 €	Received
2	Bob Smith	Google 10 Pixel Phone	800.00 €	Customer	Good	100%	800.00 €	Received
3	Charlie Davis	Lenovo Tab M12	500.00 €	Customer	Damaged	85%	425.00 €	Received
4	Hans Mustermann	Iphone 13	1200.00 €	Production	Damaged	100%	1200.00 €	Received
5	David Wilson	Google Pixel 8 Phone	300.00 €	Unknown	Not_Received	0%	0.00 €	Pending_Return

The result table demonstrates that process status, refund rate and refund amount can be stored and retrieved together, providing a traceable link between the operational decision and its financial consequence.

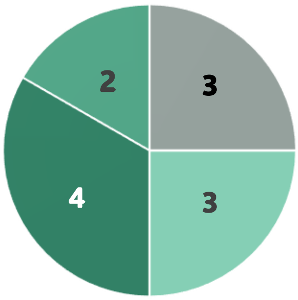
05 / RESULTS AND VALIDATION

What the academic prototype produced

The project produced verifiable modeling and data outputs. It also identifies potential operational improvements that still require real-world measurement.

12	9	3	EUR 5,544.50
Claims in the test dataset	Cases marked Received	Cases marked Pending Return	Calculated total refunds

Automated outcome distribution



- 100% refund: 4 cases
- 85% refund: 2 cases
- 70% refund: 3 cases
- 0% / pending: 3 cases

Result type	Status	Evidence
End-to-end process visibility	Produced	Initial and optimized BPMN models
Consistent refund calculation	Validated on test data	Excel formulas and structured output
Pending-return identification	Validated on test data	3 records automatically classified Pending_Return
Structured reporting base	Produced	Relational fields and SQL result table
Processing-time reduction	Indicative	Portfolio estimate of approximately 35%; not measured in production
Improved communication	Expected qualitative effect	Shared model and data structure; requires operational validation

Interpretation of the outcome

The strongest result is control, not speed alone. The redesigned model makes the logic auditable: every refund can be traced back to defined process inputs, while pending and exceptional cases remain visible instead of disappearing from the standard flow.

06 / DELIVERABLES AND CONCLUSION

A reusable foundation for implementation

Together, the deliverables connect process design, operational data and financial rules in a format that can be reviewed and extended.

Deliverable	Contribution to the project
Detailed process description	Documents on the initial operating situation, decision logic and problem statement.
Initial BPMN 2.0 model	Makes the original activities, departments and exceptional branches visible.
Optimized BPMN 2.0 model	Shows explicit refund paths, status outcomes and improved information flow.
Excel prototype	Structures claim records and calculate rate, amount and process status.
SQL script and result	Provides a relational implementation and traceable query output.
Case-study page	Communicates the context, approach, tools, outcomes and downloadable resources.

Recommended next steps

- Validate every refund-rule combination with the responsible business and legal stakeholders.
- Define process owners, service-level targets and escalation responsibilities for each status.
- Implement a deadline field and automated alert for the 30-day return window.
- Replace test names and products with anonymized production data for a controlled pilot.
- Measure lead time, error rate, rework and customer notification time before and after implementation.

Limitations

This is an academic prototype. The dataset contains 12 test cases and does not demonstrate production-scale performance. The approximate 35%-time reduction and qualitative communication improvement shown on the portfolio page are improvement hypotheses until confirmed through an operational pilot.

Conclusion

The project transforms claims management from a partly implicit sequence of handovers into an explicit, data-supported process. BPMN provides shared visibility, Excel validates the business logic and SQL establishes a scalable structure for consistent storage and retrieval.

The resulting foundation is suitable for controlled implementation, measurement and continuous improvement.